

Principles of Programming Languages, 2025.02.06

Important notes

- Total available time: 2h (*multichance* students do not need to solve Exercise 1).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.

Exercise 1, Scheme (11 pts)

Consider the *named let* construct of Scheme. Implement it as a *nlet* macro without using the built-in version of the construct.

Exercise 2, Haskell (11 pts)

A simply-linked circular list (called *Clist* from now on) is a list in which the last node points to the first node. It is sometimes useful to have a “sentinel” last node, i.e. a node that does not contain data. The sentinel is used e.g. to check if we have traversed the whole list. An empty list contains only the sentinel node, that points to itself.

Consider the following implementation of circular lists:

```
data Clist a = Node a (Clist a) | End (Clist a) -- End is the Sentinel node
```

```
instance (Show a) => Show (Clist a) where
    show (End _) = "... "
    show (Node v next) = show v ++ ", " ++ show next
```

```
instance (Eq a) => Eq (Clist a) where
    End _ == End _ = True
    (Node x next) == (Node x' next') = (x == x') && next == next'
    _ == _ = False
```

1) Explain why *Clist* does not derive *Show* and *Eq* in its definition.

2) Make *Clist* an instance of *Functor* and *Applicative*. NB: you cannot translate *Clist* to standard lists and back, in the implementation of the requested methods.

Exercise 3, Erlang (11 pts)

Consider the set of all non-null prefixes of a list (e.g. for [1, 2, 3] the prefixes are {[1], [1, 2], [1, 2, 3]}).

Write a function *check_pref* which takes two lists, *T* and *L*, then creates a set of processes *P* for each prefix of the list *L* – such processes must receive a list and control if it matches the one associated to the process: if yes, it must return the atom *yes*, otherwise the atom *no*. Then, *check_pref* uses the processes in *P* to check if *T* is a prefix of *L*.

Solutions

Ex 1

```
(define-syntax nlet
  (syntax-rules ()
    ((_ name ((x v) ...) body ...)
     (begin
       (define (name x ...)
         body ...)
       (name v ...))))))
```

Ex 2

1) The standard implementations of show and == do not terminate.

2)

instance Functor Clist where

```
fmap f (Node x next) = let first = Node (f x) $ fmap' f next first
                        in first where
                          fmap' f (End x) first = End first
                          fmap' f (Node x next) first = Node (f x) $ fmap' f next first
```

instance Applicative Clist where

```
pure x = let first = Node x $ End first
          in first
(Node f fs) <*> (Node x xs) = let first = Node (f x) $ apply' fs xs first
                              in first where
                                apply' (End t) _ first = End first
                                apply' _ (End t) first = End first
                                apply' (Node f fs) (Node x xs) first = Node (f x) $ apply' fs xs first
```

Ex 3

```
pre([], R) -> R;
pre([X|Xs], []) -> pre(Xs, [[X]]);
pre([X|Xs], [R|Rs]) -> pre(Xs, [R ++ [X] | [R|Rs]]).
prefix(L) -> pre(L, []).
```

check_list(L, Master) ->

```
  receive
    L -> Master ! yes;
    _ -> Master ! no
  end,
  check_list(L, Master).
```

check_pref(T, L) ->

```
  I = self(),
  Pids = [spawn(fun () -> check_list(X, I) end) || X <- prefix(L)],
  [P ! T || P <- Pids],
  lists:member(yes, [receive X -> X end || _ <- Pids]).
```